PRIVATE SPORTS COURT RESERVATION SYSTEM LOCATED IN SAN JOSE, PUERTO PRINCESA CITY, PALAWAN

Technical Documentation and System Integration Paper

Project Title: Private Sports Court Reservation System Located in San Jose, Puerto Princesa City, Palawan

Course: BSIT

Prepared by:

Ariel James Adesir

Fea Bulanay

Johnruzzel Correa

Jerome Gatila

Jenny Benosa

Instructor: Toni Rose Correa

October 20, 2026

Chapter 1

INTRODUCTION

1.1 Project Overview

The Private Sports Court Reservation System is an organized platform that allows users to reserve a court at a private sports facility. Through an Availability Calendar, users can check each court's status (Available, Reserved, or Pending) for a selected date and time, choose their preferred court and schedule, specify the reservation duration, and submit their booking. Reservations are completed through online payment, after which the system sends a confirmation and receipt to the user's registered email address as proof of reservation. This allows users to keep a copy of their receipt even when they have no mobile load or need it at the last minute. The system is intended for the private sports court located in Barangay San Jose, Puerto Princesa City, Palawan.

Traditional reservation methods require users to visit the facility or wait for an administrator to confirm availability. Manual processes can lead to double bookings, delayed confirmations, and disorganized records of reservations, cancellations, schedules, and payments, making information harder to verify and manage.

The system addresses these challenges by providing a centralized platform and database for managing court information, schedules, reservations, users, and payment records. Its availability calendar and status indicators help prevent conflicting reservations, while email notifications give users a quick and accessible record of their transactions. It is intended for privately owned and operated sports courts, offering a more convenient, organized, and reliable approach to sports court reservation.

1.2 Problem Statement

Manual court reservation in private sports facilities is inconvenient and prone to errors. Users must visit the facility or contact the administrator just to check availability, which can lead to double bookings, delayed confirmations, and disorganized reservation and payment records.

The Private Sports Court Reservation System addresses these problems by allowing users to check court availability, reserve a court, and pay online or on-site, with an email confirmation and receipt, while providing administrators with a centralized system to manage courts, schedules, reservations, and payments.

1.3 Project Objectives

General Objective

The general objective of the project is to develop an efficient, organized, and centralized Private Sports Court Reservation System for a private sports facility that provides users with a convenient reservation process while simplifying the management and monitoring of court availability, schedules, reservations, and payment records for administrators.

Specific Objectives

Specifically, the system aims to:

- Provide an Availability Calendar that shows the Available, Reserved, and Pending status of the private courts.
- Allow users to reserve a court by selecting the date, time, and duration, while preventing double reservations.
- Provide online and on-site payment options for court reservations.
- Send reservation confirmations and receipts to users' registered email addresses.
- Provide a downloadable payment receipt right after payment, so users can save proof of booking even without mobile load or when they need it at the last minute.
- Provide administrators with tools to manage courts, schedules, reservations, and payment records in a centralized and secure database.

Chapter 2

System Analysis & Requirements Specification

The system aims to provide a convenient, organized, and reliable way for customers to reserve private sports courts online. It is designed to reduce the need for manual reservations, prevent overlapping court schedules, simplify payment monitoring, and help administrators manage court availability and reservation records efficiently.

2.1 Functional Requirements

FR-1 User Registration and Login

FR-1.1: Customer must create an account and register.

FR-1.2: Customer must log in using the registered account before reserving a court.

FR-2 Availability Calendar

FR-2.1: System must display the status of each court (Available, Reserved, or Pending) per date and time.

FR-3 Court and Schedule Selection

FR-3.1: Customer must choose a court.

FR-3.2: Customer must choose the date, start time, and duration of the reservation.

FR-3.3: System must check the schedule and reject any reservation that overlaps with an existing Reserved or Pending reservation.

FR-4 Payment System and Method Selection

FR-4.1: Customer must select a payment method: Online Payment or On-site Payment.

FR-4.2: For online payment, the customer must pay through the Payment Provider, and the reservation is confirmed only after the payment is verified.

FR-4.3: For on-site payment, the system must hold the schedule as Pending and automatically cancel it if unpaid within the payment period.

FR-4.4: The administrator must verify on-site payments and update the reservation and payment status.

FR-5 Receipt and Email Confirmation

FR-5.1: System must generate a downloadable receipt (reference no., court, date, time, duration, amount, and status) in PDF or image format right after payment.

FR-5.2: For on-site payments, system must generate the receipt, make it downloadable, and email it after the administrator verifies the payment.

FR-5.3: System must send the confirmation and receipt to the customer's registered email address.

FR-5.4: System must not generate receipts for Pending, unpaid, or cancelled reservations.

FR-5.5: Receipt must remain downloadable in the reservation records even if the email fails.

FR-6 Reservation Status and History

FR-6.1: Customer must be able to view the status and history of their reservations.

FR-6.2: Customer must be able to re-download receipts of paid reservations.

FR-7 Court Management

FR-7.1: Administrator must be able to add, edit, and update courts (name, location, availability, and rates).

FR-7.2: Administrator must be able to mark courts as unavailable (maintenance, repairs, or private events).

FR-8 Reservation and Payment Monitoring

FR-8.1: Administrator must be able to monitor and update reservation records and status.

FR-8.2: Administrator must be able to review the payment method, amount, date, and status of each payment.

FR-9 Reports

FR-9.1: System must generate reservation and payment reports for the administrator.

2.2 Non-Functional Requirements

Security: the system protects accounts and records through authentication and role-based access, so customers can see only their own reservations and receipts, and only the administrator can open the admin pages.

Performance: the system responds efficiently to requests such as loading the availability calendar and submitting a reservation.

Data validation: every form is checked for missing or wrong input before anything is saved. Incomplete or invalid reservations are rejected.

Data integrity: reservation and payment records must stay accurate and consistent, and a payment must not be counted twice or confirm a reservation that overlaps another.

Usability: the system must be easy for customers to use, including on mobile devices.

Maintainability: the system is organized so that developers can maintain and update it.

Scalability: the system can handle more courts, customers, and reservations over time without changing its structure.

Backup and recovery: the system provides backup and recovery so that reservations, payments, and accounts are not lost.

Receipt accessibility: receipts are downloadable as lightweight formats (PDF or image), so they can be saved and used even with limited internet connectivity.

2.3 System Use Cases

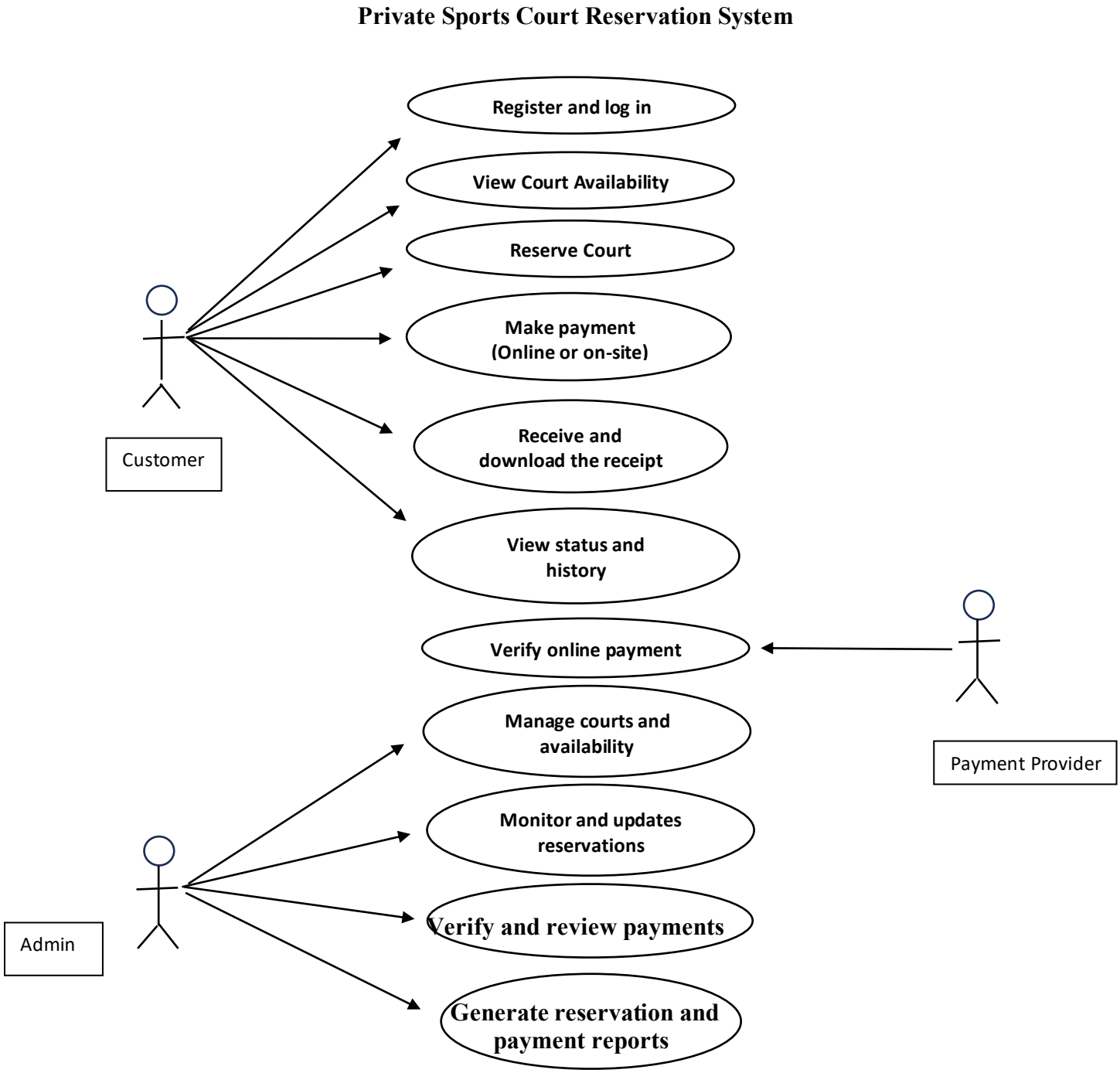


Figure 1 shows the use case diagram of the Private Sports Court Reservation System. It has three actors: the Customer, the Administrator, and the Payment Provider. All use cases are placed inside the system boundary.

Use Case ID	Use Case Title	Sub-Use Case ID	Description
SUC-1	Register and Login	SUC-1.1	Customer creates an account and registers so that every reservation is linked to the right person.
SUC-1	Register and Login	SUC-1.2	Customer logs in using the registered account before reserving a court.
SUC-2	View Court Availability	SUC-2.1	Customer views the Availability Calendar to see which courts are free before booking, without visiting the facility or contacting the administrator.
SUC-2	View Court Availability	SUC-2.2	System displays the status of each court (Available, Reserved, or Pending) per date and time.
SUC-3	Reserved Court	SUC-3.1	Customer reserves a court online by choosing the court, date, start time, and duration.
SUC-3	Reserved Court	SUC-3.2	System checks the schedule and rejects any reservation that overlaps an existing Reserved or Pending reservation to prevent double bookings.
SUC-4	Make payment (online or on-site)	SUC-4.1	Customer completes the reservation by selecting a payment method: Online Payment or On-site Payment.
SUC-4	Make payment (online or on-site)	SUC-4.2	For online payment, the customer pays through the Payment Provider, and the reservation is confirmed only after the payment is verified.
SUC-4	Make payment (online or on-site)	SUC-4.3	For on-site payment, the system holds the schedule as Pending and automatically cancels it if unpaid within the payment period.
SUC-5	Receive and Download Receipt	SUC-5.1	Customer receives proof of reservation that can be saved even without mobile load or when it is needed at the last minute.
SUC-5	Receive and Download Receipt	SUC-5.2	System generates a downloadable receipt (PDF or image) and emails the confirmation and receipt to the customer's registered email address once payment is confirmed.
SUC-5	Receive and Download Receipt	SUC-5.3	Receipt remains downloadable in the reservation records even if the email fails. System does not generate receipts for Pending, unpaid, or canceled reservations.
SUC-6	View Status and History	SUC-6.1	Customer tracks reservations by viewing their status and history.

SUC-6	View Status and History	SUC-6.2	Customer re-downloads receipts of paid reservations.
SUC-7	Verify Online Payment	SUC-7.1	Payment Provider verifies the online payment to make sure it is valid before the reservation is confirmed.
SUC-7	Verify Online Payment	SUC-7.2	Payment Provider sends the result back to the system, which confirms the reservation only if the payment is verified.
SUC-8	Manage Courts and Availability	SUC-8.1	Administrator keeps court information accurate by adding, editing, and updating courts (name, location, availability, and rates).
SUC-8	Manage Courts and Availability	SUC-8.2	Administrator marks courts as unavailable for maintenance, repairs, or private events.
SUC-9	Monitor and Update Reservations	SUC-9.1	Administrator monitors reservation records to keep them organized.
SUC-9	Monitor and Update Reservations	SUC-9.2	Administrator updates the reservation status when needed.
SUC-10	Verify and Review Payments	SUC-10.1	Administrator verifies on-site payments and updates the reservation and payment status.
SUC-10	Verify and Review Payments	SUC-10.2	Administrator reviews the payment method, amount, date, and status of each payment.
SUC-10	Verify and Review Payments	SUC-10.3	After an on-site payment is verified, system generates the receipt and emails it to the customer.
SUC-11	Generate Reservation and Payment Reports	SUC-11.1	System generates reservation and payment reports so the administrator can monitor reservations and payments.

Chapter 3

Architecture and System Design

This section explains how the Private Sports Court Reservation System is built. It presents the technologies used, how the parts of the system are connected, and how the system communicates with the Payment Provider and the Email Service during a reservation.

3.1 Technical Stack Justification

The system is a web-based application. It was built with open-source technologies and one third-party payment service. Each technology was chosen because it fits the requirements in Chapter 2, which are secure accounts, a reservation process that prevents double bookings, online payment that is verified before a reservation is confirmed, and downloadable receipts sent by email. Table 1 summarizes the stack, and the paragraphs after it explain the reason for each choice.

Table 1

Technical stack of the Private Sports Court Reservation System

Layer	Technology	Reason for selection
Frontend	HTML forms and pages rendered by Jinja2 templates	Pages are produced by the server, so no separate client application is needed. Jinja2 escapes displayed text automatically, which protects against cross-site scripting. Pages can be displayed in mobile browsers, which supports the usability requirement.

Layer	Technology	Reason for selection
Backend language and framework	Python with the Flask web framework	Flask is lightweight and gives the developers direct control over routes, sessions, and business rules. Python also supports the payment, scheduling, and email libraries used by the system.
Database	MySQL, accessed through Flask-SQLAlchemy and the PyMySQL driver	Reservations, payments, and receipts are related records that must stay consistent, so a relational database with transactions and row locking is suitable. SQLAlchemy keeps user input separate from SQL commands, which prevents SQL injection.
Payment processor	PayMongo Checkout Sessions API (card, GCash, and Maya)	PayMongo is a payment gateway for the Philippines that supports the payment methods Filipino customers commonly use. Its hosted checkout page receives the card or e-wallet details, so the system never handles them. It also sends signed webhooks for server-side verification.
Email service	Flask-Mail with an SMTP server (app password)	SMTP is a standard way to send email, and no additional paid service is required. The email carries the confirmation and the PDF receipt as a record of the reservation.

Layer	Technology	Reason for selection
Security and scheduling libraries	Flask-WTF, Flask-Limiter, APScheduler	Flask-WTF adds CSRF tokens to forms, and Flask-Limiter limits repeated login and registration attempts. APScheduler runs the background jobs that cancel unpaid Pending reservations and re-check online payments.

Programming language and framework

The backend is written in Python using the Flask framework. Flask is a micro-framework, which means it provides the core parts of a web application, such as routing, request handling, and sessions, and leaves the other parts to extensions. This suits a system with a limited scope, such as one private sports facility, because the team adds only what the requirements need. Python was also chosen because the libraries used for payment requests, PDF receipts, email, and scheduled jobs are available for it, and the language is readable enough for the developers to maintain and update the system, as required in Chapter 2.

Frontend

The user interface is made of HTML forms and pages that Flask generates with the Jinja2 template engine. Because the pages are produced by the server, the business rules and the access checks stay in one place, the backend, and the browser only displays the result. Jinja2 escapes displayed text automatically, and every form carries a hidden CSRF token. The pages are meant to work in the browsers of both desktop and mobile devices, since customers are expected to reserve a court from their phones.

Database

MySQL is the database engine. The data of the system is structured and related: a reservation belongs to a customer and a court, a payment belongs to a reservation, and a receipt belongs to a paid reservation. A relational database represents these relationships directly. It also supports transactions and row locking. These are needed to meet the data integrity requirement, because the court record is locked while the system checks for overlapping reservations, and a reservation and its payment record are saved together or not at all. Flask-SQLAlchemy and the PyMySQL driver connect the application to MySQL. SQLAlchemy sends user input separately from the SQL command, so every query is parameterized.

Third-party payment service: PayMongo

PayMongo is the Payment Processor of the system. It was selected for three reasons. First, it supports cards, GCash, and Maya, which allows customers to pay with the methods they already use. Second, its hosted checkout page collects the card or e-wallet details, so the system never receives or stores them, and the Payments table has no field for card data. This keeps the system away from the handling of card data described by the PCI Security Standards Council (2024). Third, PayMongo sends a webhook, a server-to-server message signed with a secret, when a payment succeeds or fails. The system can then confirm a reservation without trusting the customer's browser. The backend communicates with PayMongo through its REST API using the Python requests library.

Email service

Confirmation and receipts are sent with Flask-Mail through an SMTP server, authenticated with an app password. SMTP was chosen because it is a standard and widely supported protocol that does not require an additional paid service. The email is sent only after the database is saved, so an email problem cannot undo a confirmed

payment. A failed email is marked as Failed and tried again later, and the receipt remains downloadable from the reservation history.

Supporting libraries

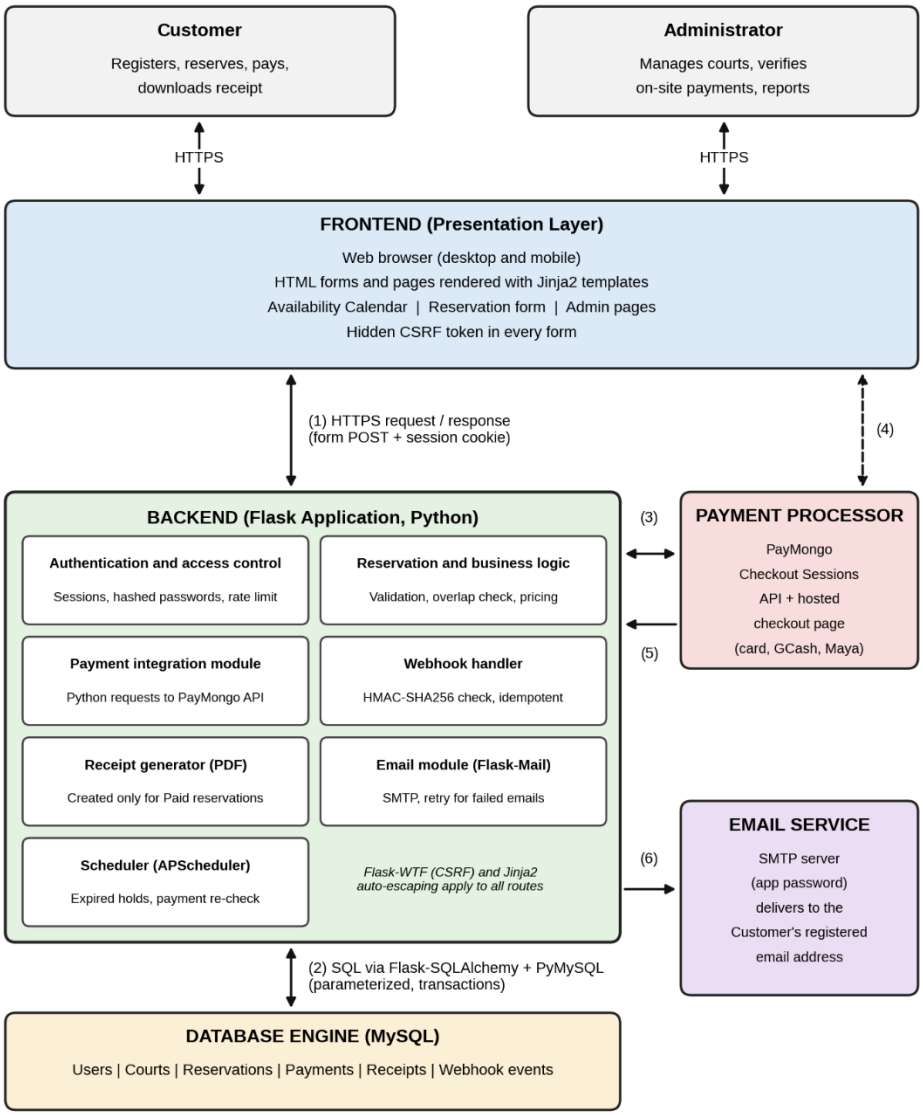
Flask-WTF provides the CSRF tokens in the forms, and Flask-Limiter rate-limits the login and registration routes. APScheduler runs scheduled jobs inside the application. One job cancels unpaid Pending reservations when the hold time ends, and another re-check Pending online payments with PayMongo in case a webhook does not reach the system. These choices follow the security practices of the OWASP Application Security Verification Standard (OWASP Foundation, 2025), which are discussed further in Chapter 4.

3.2 Architectural Block Diagram

Figure 2 shows how the main parts of the system are connected. The system has five parts: the Frontend, the Backend, the Database Engine, the Payment Processor (PayMongo), and the Email Service. The numbers in the figure match the numbered interactions that are explained below.

Figure 2

Architectural block diagram of the Private Sports Court Reservation System



Frontend. The Customer and the Administrator use a web browser to open the pages of the system. The Frontend displays the Availability Calendar, the reservation form, and the administrator pages, and it sends form data to the Backend.

Backend. The Flask application is the center of the system. It checks the login, the role, and the CSRF token of every request, validates the input, applies the reservation rules, computes the amount, and communicates with the other parts. Its modules are

authentication and access control, reservation and business logic, payment integration, the webhook handler, the PDF receipt generator, the email module, and the scheduler. Database Engine. MySQL stores the Users, Courts, Reservations, Payments, Receipts, and Webhook events tables. The Backend is the only part that reads or writes the database; the browser, PayMongo, and the email server never connect to it directly. Payment Processor. PayMongo creates the checkout session, hosts the payment page, processes the payment, and reports the result to the Backend. Email Service. The SMTP server receives the confirmation and the receipt from the Backend and delivers them to the registered email address of the Customer.

The interactions in Figure 2 are as follows:

1. The browser and the Backend communicate over HTTPS. The browser sends form data by POST with the session cookie, and the Backend answers with pages or redirects.
2. The Backend reads and writes MySQL through Flask-SQLAlchemy and PyMySQL, using parameterized queries and transactions.
3. The Backend sends requests to the PayMongo REST API to create a checkout session and to retrieve its payment status, and PayMongo answers with the result.
4. The Customer is redirected from the Backend to the PayMongo-hosted checkout page and then back to the return page of the system. Payment details are entered only on the PayMongo page.
5. PayMongo sends a signed webhook to the webhook route of the Backend. This is the main way a payment is confirmed.
6. After a payment is confirmed, the Backend generates the PDF receipt and sends it to the Email Service through SMTP.

The diagram also shows where the system is protected. The Customer's payment details do not pass through the Backend; the webhook is accepted only when its signature is valid, and the receipt email is sent only after the confirmed payment is saved in the database. For the webhook to reach the Backend, the system must be reachable from the internet through an HTTPS address. During testing, a temporary public URL provides this.

3.3 Sequence Diagrams

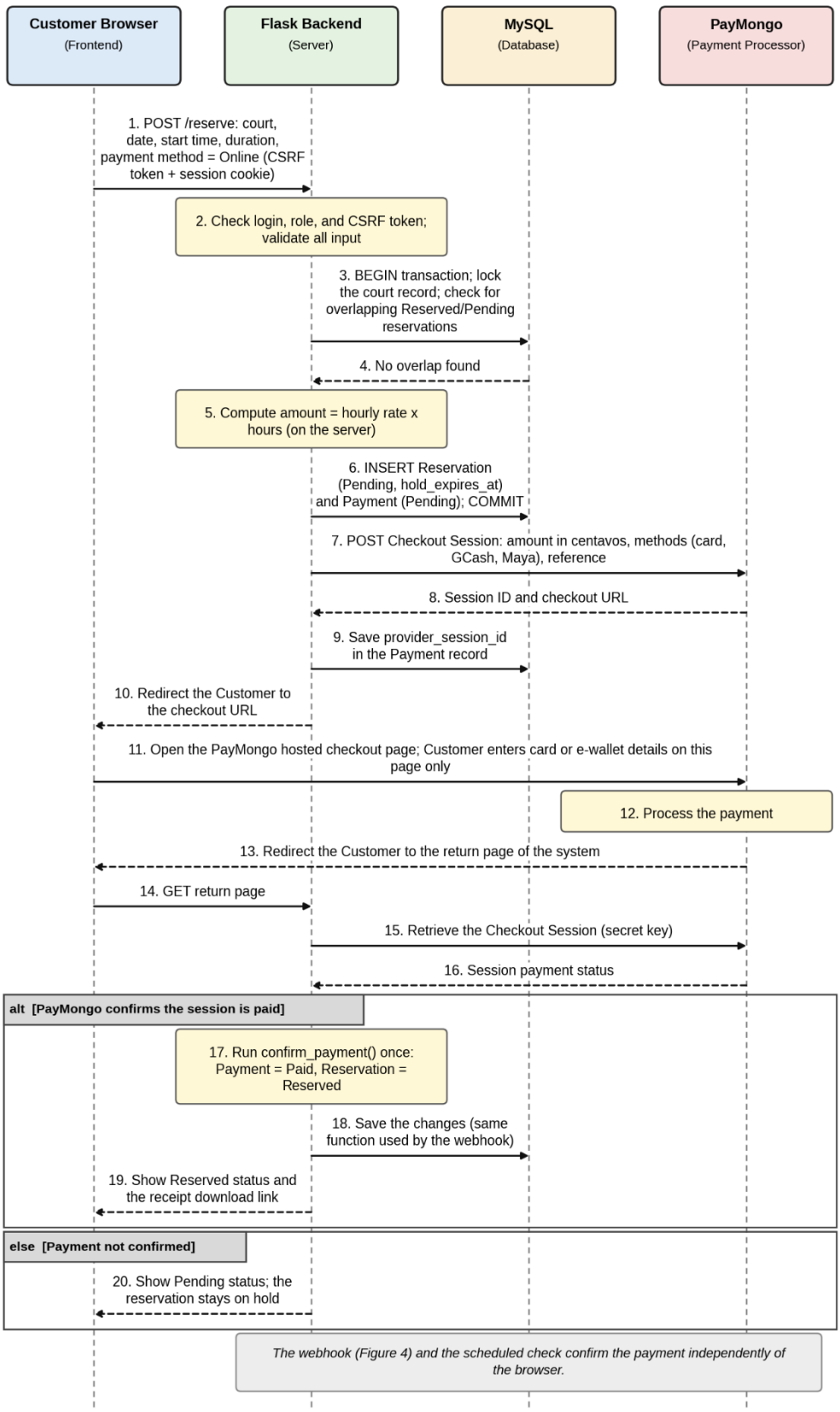
The sequence diagrams show the order of the messages exchanged between the parts of the system for the two main parts of an online payment: the payment checkout and the webhook processing. Solid arrows are requests, dashed arrows are responses, and the shaded boxes are actions done by one part itself.

3.3.1 Payment checkout

Figure 3 shows the flow from the submission of the reservation form until the Customer returns from PayMongo.

Figure 3

Sequence diagram of the online payment checkout



The checkout works as follows. The Customer submits the reservation form with Online Payment selected (step 1). The Backend checks the login, the role, and the CSRF token, and validates the input (step 2). Inside a database transaction, it locks the court record and checks for an overlapping Reserved or Pending reservation (steps 3 and 4). It then computes the amount on the server from the hourly rate and the number of hours (step 5) and saves the reservation and its payment record as Pending, which blocks the schedule for other customers (step 6).

Next, the Backend sends a request to the PayMongo Checkout Sessions API with the amount in centavos, the allowed payment methods, and the reservation reference (step 7). PayMongo answers with a session ID and a checkout URL (step 8). The Backend saves the session ID (step 9) and redirects the Customer to the checkout URL (step 10). The Customer enters the card or e-wallet details on the PayMongo page, and PayMongo processes the payment (steps 11 and 12).

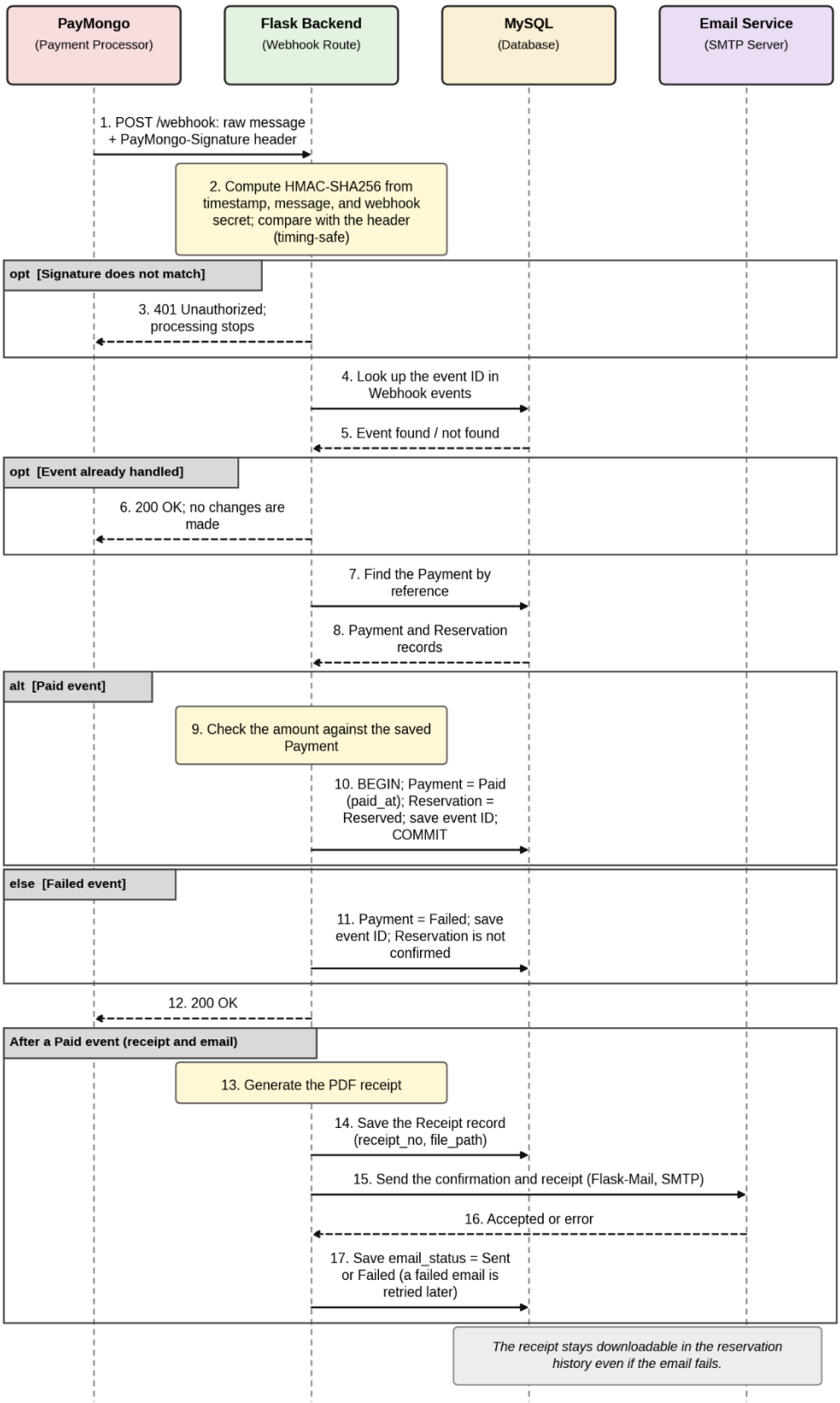
PayMongo then redirects the Customer to the return page of the system (steps 13 and 14). The return page does not trust the browser. The Backend asks PayMongo directly, using the secret key, whether the checkout session was paid (steps 15 and 16). If PayMongo confirms it, the Backend runs the confirmation function, which sets the payment to Paid and the reservation to Reserved, and shows the Customer the receipt download link (steps 17 to 19). If PayMongo does not confirm it, the reservation stays Pending (step 20). Because the confirmation function changes a payment only once, it is safe to run it from the return page, the webhook, and the scheduled job.

3.3.2 Webhook processing

Figure 4 shows how the Backend processes the message that PayMongo sends after a payment succeeds or fails. This message is independent of the browser, so the reservation is confirmed even if the Customer closes the page after paying.

Figure 4

Sequence diagram of the webhook processing, receipt generation, and email sending


PayMongo
(Payment Processor)
Flask Backend
(Webhook Route)
MySQL
(Database)
Email Service
(SMTP Server)
1. POST /webhook: raw message + PayMongo-Signature header
2. Compute HMAC-SHA256 from timestamp, message, and webhook secret; compare with the header (timing-safe)
opt [Signature does not match]
3. 401 Unauthorized; processing stops
4. Look up the event ID in Webhook events
5. Event found / not found
opt [Event already handled]
6. 200 OK; no changes are made
7. Find the Payment by reference
8. Payment and Reservation records
alt [Paid event]
9. Check the amount against the saved Payment
10. BEGIN; Payment = Paid (paid_at); Reservation = Reserved; save event ID; COMMIT
else [Failed event]
11. Payment = Failed; save event ID; Reservation is not confirmed
12. 200 OK
After a Paid event (receipt and email)
13. Generate the PDF receipt
14. Save the Receipt record (receipt_no, file_path)
15. Send the confirmation and receipt (Flask-Mail, SMTP)
16. Accepted or error
17. Save email_status = Sent or Failed (a failed email is retried later)
The receipt stays downloadable in the reservation history even if the email fails.

PayMongo sends the raw message and the PayMongo-Signature header to the webhook route (step 1). The Backend computes an HMAC-SHA256 signature from the timestamp, the message, and the webhook secret, and compares it with the header using a timing-safe function (step 2). If the signatures are different, the Backend answers 401 and stops (step 3). The route has no CSRF token because PayMongo cannot send one, so the signature protects it instead.

If the signature is valid, the Backend looks up the event ID in the Webhook events table (steps 4 and 5). When the event was already handled, the Backend answers 200 OK and changes nothing, so a repeated webhook cannot confirm a reservation twice (step 6). Otherwise, it finds the payment by its reference (steps 7 and 8). For a paid event, it checks the amount and, in one transaction, sets the payment to Paid, the reservation to Reserved, saves the payment time and the event ID (steps 9 and 10). For a failed event, it sets the payment to Failed, and the reservation is not confirmed (step 11). The Backend then answers 200 OK (step 12).

After a paid event, the Backend generates the PDF receipt and saves the receipt record (steps 13 and 14). It sends the confirmation and the receipt to the Email Service through SMTP (steps 15 and 16) and records the email status as Sent or Failed (step 17). The email is sent after the database is saved, so a mail problem cannot undo a confirmed payment. A failed email is tried again later, and the Customer can still download the receipt from the reservation history.

For an on-site payment, no checkout session or webhook is used. The reservation stays Pending until the Administrator verifies the payment. The same confirmation function then sets the payment to Paid and the reservation to Reserved, and the receipt and email steps (13 to 17) follow.

Chapter 4

Implementation Details

This chapter explains how the system is implemented: its data operations and business rules, the database connection, payment and Email integration, and the application is protected.

4.1 CRUD & Business Logic Module

Create. The Customer creates an account through the registration form and creates a reservation by choosing a court, a date, a time, and the payment method. The Administrator adds courts and sets their hourly rates. Every form is checked first, and nothing is saved when the input is wrong.

Read. The Customer sees the Availability Calendar of a court (Available, Pending, or Reserved) and the history of his or her own reservations and receipts. The Administrator sees all reservations and payments and can generate reports. Each query for a customer also checks the Customer's ID, so a customer cannot open another Customer's reservation by changing the number in the address.

Update. The Administrator can edit court details and rates, mark a court as unavailable, update the status of a reservation, and verify on-site payments. A Customer cannot change a reservation that is already paid.

Delete. Records are not deleted for good. A cancelled, expired, or unpaid reservation is marked Cancelled, and a court that is no longer used is marked unavailable. This keeps the history complete for monitoring and reports.

Price computation. The system computes the amount from the hourly rate of the court and the number of hours:

Amount = hourly rate x number of hours.

The server computes the amount again when the reservation is saved, so the saved amount never depends on what the browser sent

Table 1

Worked example of reservation amount

Item	Computation	Amount (PHP)
Hourly rate of the court	Set by the Administrator	300.00
Reservation of 2 hours	300.00 x2	600.00
Amount sent to PayMongo	600.00 x 100	60.000

Reservation rules. The system follows these rules:

- A reservation is rejected when it overlaps with a Reserved or Pending reservation of the same court at the same time. The court is locked during the check, so two requests at the same time cannot both succeed.
- A new reservation is saved as Pending, which blocks the schedule for other Customers. A scheduled job cancels unpaid Pending reservations when the hold time ends.
- A reservation becomes Reserved only when the payment is verified, by the webhook or by the Administrator.
- A receipt is made only for a Paid reservation, never for a Pending, unpaid, or cancelled one.

Validation rules. The system checks the following before saving:

- Names are not empty, emails have a correct format, and passwords have at least 8 characters and match the confirmation.

- The court, date, start time, and duration must be chosen. The date and time cannot be in the past, and the duration must be a positive number of whole hours.
- Court names are not empty, and court rates must be positive numbers.

Frontend-to-backend data flow. Every page that changes data follows the same steps:

1.The user fills in an HTML form. The form carries a hidden CSRF token.

2.The browser sends the form to a Flask route by POST, together with the session cookie.

3.The route checks the login, the role, and the token.

4.The route validates the input. When the input is wrong, it shows the form again with a message, and nothing is saved.

5.When the input is correct, the route applies the rules and saves the data with SQLAlchemy, which uses parameterized queries, inside a transaction.

6.The route redirects to a list page and shows a short confirmation message.

4.2 Database Connectivity

The backend connects to MySQL with Flask-SQLAlchemy and the PyMySQL driver. The host, user, password, and database name are stored in environment variables, and the entire application uses a single shared database object that maintains a pool of open connections. SQLAlchemy sends user input separately from the SQL command, so every query is parameterized. Creating a reservation uses a transaction: the court record is locked while the system checks for overlaps, and the reservation and its payment record are saved together or not at all. MySQL handles data persistence.

Nothing important is kept only in variables, so the data is still there after the server restarts.

Table 2.

Database tables

Table	Purpose	Main fields
Users	Login accounts of the Administrator and the Customers	Name, email, password (hash), role
Courts	Courts, rates, and availability	Name, location, hourly_rate, is_available
Reservations	One reservation for a court for a date and time	Reference_no, user_id, court_id, start_time, end_ time, amount, status, hold_expires_at
Payments	Payment of a reservation	Reservation_id, method (online or onsite), provider_session_id, amount, status (pending, paid, failed), paid_at
Receipts	Generated PDF receipt	Reservation_id, receipt_no, file_path, email_status
Webhook events	Webhook events that were already handled	Event_id, event_type, received_at

The payments table does not have any field for a card number, card owner, or security code. The webhook events table saves each event ID once, so a repeated webhook can be recognized. Regular backups of the database are made with scheduled dumps.

4.3 Subsystem Integration (Payment & Email)

Payment checkout. When the Customer chooses online payment, the backend reads the reservation of the Customer, computes the amount, and sends a request (with the Python requests library) to the PayMongo Checkout Sessions API with the total in centavos, the allowed payment methods (card, GCash, and Maya), and the reservation reference. PayMongo answers with a session ID and a checkout URL. The backend saves a payment record with the status Pending and sends the Customer to the

PayMongo page. Card or e-wallet details are typed only on the PayMongo page (PCI Security Standards Council, 2024).

Webhook. The reservation is never marked as Reserved only because the Customer opens the return page of the website; the Customer could open that page without paying. The main way to confirm a payment is the webhook: PayMongo sends a server-to-server message to the webhook route of the system. The route works in this order:

1. It reads the raw message and the PayMongo-Signature header.
2. It computes an HMAC-SHA256 signature from the timestamp, the message, and the webhook secret, and compares it with the header using a timing-safe function. If they are different, it answers 401 and stops. This route has no CSRF token, because PayMongo cannot send one, so the signature protects it instead.
3. It finds the payment by the reference. If the event was already handled, it does nothing, so a repeated webhook cannot confirm a reservation twice.
4. For a paid event, it checks the amount, sets the payment to Paid and the reservation to Reserved, and saves the time. For a failed event, it sets the payment to Failed, and the reservation is not confirmed.
5. It answers 200 OK, and then the receipt is made, and the email is sent.

Return page check. When the Customer comes back from PayMongo, the return page does not trust the browser. The server asks PayMongo directly, using the secret key, if the checkout session was paid. If it was paid, the page runs the same confirmation function as the webhook. If PayMongo does not confirm it, the reservation

stays Pending. A scheduled job (APScheduler) does the same check for Pending online reservations, which covers the case when the webhook cannot reach the system. Because the confirmation function changes a payment only once, using it from both ways is safe.

On-site payment. The reservation stays Pending, and no receipt is made. When the Administrator verifies the payment, the same function sets the payment to Paid and the reservation to Reserved. If the Customer does not pay within the payment period, a scheduled job marks the reservation as Cancelled, and the schedule becomes Available again.

Receipt and email. After a payment is confirmed, the system makes a PDF receipt with the reference number, court, date, time, duration, amount, and status. The receipt is sent by email with Flask-Mail through SMTP using an app password. The email is sent after the database is saved, so a mail problem cannot undo a confirmed payment. A failed email is marked as Failed and tried again later, and the Customer can still download the receipt from the reservation history.

4.4 Application Security

The main security measures and the problem each one prevents. The measures follow the OWASP Application Security Verification Standard (OWASP Foundation, 2025) and the NIST Secure Software Development Framework (National Institute of Standards and Technology, 2022).

Table 4

Security measures

Concern	Measure in the system
Login	User logs in with email and password. The session uses a signed cookie that expires after a set time
Access Control	Every protected page checks the user's login and role. Customers see only their own data.
Stolen Passwords	Passwords are saved as hashes, never as plain text
SQL injection	SQLAlchemy keeps user input separate from SQL commands
Cross-site scripting	Jinja2 automatically escapes displayed text
Forged request (CSRF)	Forms use CSRF tokens (Flask-WTF)
Repeated login attempts	Login and registration are rate-limited (Flask-Limiter)
Changed Prices	The amount is computed and checked on the server
Fake payment messages	The PayMongo webhook signature is verified (HMAC-SHA256)
Card data	The system never receives or stores card details
Secrets	Keys and passwords are kept in environment variables
Information leakage	Debug mode is off, and error pages are generic

Encryption and HTTPS. Passwords are protected by hashing, and the session cookie and the webhook messages are protected by signatures. Data in transit should be encrypted with HTTPS. A laptop that runs the system locally uses plain HTTP, so for real use the system should be placed on a server with an HTTPS certificate. During testing, the temporary public URL uses HTTPS to provide an encrypted connection to the local system.

CHAPTER 5

Testing and Quality Assurance

This chapter shows how the Private Sports Court Reservation System was tested to make sure that its data operations, payment flow, and security measures work as described in Chapters 2 to 4. The test cases follow the functional suitability and security characteristics of the ISO/IEC 25010 product quality model (International Organization for Standardization, 2023). Testing was done manually from the user's side, using the Customer and Administrator accounts, and the PayMongo test mode was used so that no real money was charged.

5.1 Functional Test Cases

The test cases cover the Create, Read, Update, and Delete operations and the business rules of the system. Each test case has an input, an expected outcome, and the actual result that was recorded when the test was run. A test passes when the actual result is the same as the expected outcome.

Table 5

Functional test cases for CRUD operations and data mutations

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
CREATE					
TC-01	Register a valid account	Enter a name, a correct email, and a password of at least 8 characters that matches the confirmation.	Account is saved with the Customer role, and the user is redirected to the login page with a confirmation message.	To be recorded after the actual test	Pending
TC-02	Register with a wrong email format	Enter "jen@" as the email and fill in the other fields correctly.	The form is shown again with an error message. Nothing is saved.	To be recorded after the actual test	Pending
TC-03	Register with a weak or mismatched password	Enter a 5-character password, then a password that does not match the confirmation.	Both attempts are rejected with an error message. No account is created.	To be recorded after the actual test	Pending
TC-04	Create a valid reservation	Logged-in Customer chooses an available court, a future date, a start time, 2 hours, and Online Payment.	Reservation is saved as Pending with a reference number. The schedule is blocked for other Customers. Amount is 600.00 when the hourly rate is 300.00.	To be recorded after the actual test	Pending
TC-05	Reservation in the past	Choose a date or time that has already passed.	Reservation is rejected with an error message. Nothing is saved.	To be recorded after the actual test	Pending
TC-06	Reservation that overlaps another	Choose a court and time that overlaps an existing Reserved or Pending reservation of the same court.	Reservation is rejected with a message that the schedule is not available. Nothing is saved.	To be recorded after the actual test	Pending

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
TC-07	Reservation with missing or invalid duration	Leave the court, date, start time, or duration empty. Then enter a duration of 0 and of 1.5 hours.	Each attempt is rejected with an error message. Nothing is saved.	To be recorded after the actual test	Pending
TC-08	Reserve with On-site Payment	Logged-in Customer submits a valid reservation and selects On-site Payment.	Reservation is saved as Pending with a hold time. No receipt is made.	To be recorded after the actual test	Pending
TC-09	Administrator adds a court	Administrator enters a court name, location, and an hourly rate of 300.00.	The court is saved and appears in the court list and in the Availability Calendar.	To be recorded after the actual test	Pending
TC-10	Add a court with wrong data	Leave the court name empty. Then enter an hourly rate of 0 and of -100.	Each attempt is rejected with an error message. No court is saved.	To be recorded after the actual test	Pending
READ					
TC-11	View the Availability Calendar	Open the calendar of a court for a date that has Available, Pending, and Reserved slots.	Each slot shows the correct status: Available, Pending, or Reserved.	To be recorded after the actual test	Pending
TC-12	Customer views own history	Logged-in Customer opens the reservation history.	Only the reservations and receipts of that Customer are shown, with their status.	To be recorded after the actual test	Pending
TC-13	Customer opens another Customer's reservation	Change the reservation number in the address bar to one that belongs to another Customer.	Access is denied, or the page is not found. The other reservation is not shown.	To be recorded after the actual test	Pending
TC-14	Administrator views all records	Administrator opens the reservation and payment pages.	All reservations and payments are shown with method, amount, date, and status.	To be recorded after the actual test	Pending

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
TC-15	Generate reports	Administrator generates the reservation and payment reports.	Reports are shown with data that matches the saved records.	To be recorded after the actual test	Pending
UPDATE					
TC-16	Edit court details and rate	Administrator changes the name and the hourly rate of a court.	Changes are saved. New reservations use the new rate. Old reservations keep their saved amount.	To be recorded after the actual test	Pending
TC-17	Mark a court as unavailable	Administrator marks a court as unavailable for maintenance.	The court cannot be reserved, and the calendar does not show it as Available.	To be recorded after the actual test	Pending
TC-18	Update reservation status	Administrator changes the status of a reservation.	The new status is saved and shown in the Customer's history.	To be recorded after the actual test	Pending
TC-19	Verify an on-site payment	Administrator verifies the payment of a Pending On-site reservation.	Payment becomes Paid, and the reservation becomes Reserved. A receipt is made and emailed to the Customer.	To be recorded after the actual test	Pending
TC-20	Customer edits a paid reservation	Customer tries to change a reservation that is already paid.	The change is not allowed. The reservation stays the same.	To be recorded after the actual test	Pending
DELETE (marked as Cancelled or unavailable, not removed)					
TC-21	Unpaid Pending reservation expires	Create an On-site reservation and do not pay until the hold time ends.	The scheduled job marks it Cancelled, and the schedule becomes Available again.	To be recorded after the actual test	Pending

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
TC-22	Cancelled reservation stays in the records	Open the history of the Customer and the reservation list of the Administrator after TC-21.	The cancelled reservation is still shown with the status Cancelled. No receipt is made for it.	To be recorded after the actual test	Pending
TC-23	Court that is no longer used	Administrator marks a court that has past reservations as unavailable.	The court is not deleted. Its past reservations and reports stay complete.	To be recorded after the actual test	Pending

5.2 Payment and Receipt Test Cases

Table 6 lists the test cases for the online payment flow, the webhook, the return page check, and the receipt and email. All payments were made in PayMongo test mode, so no real money was charged.

Table 6

Test cases for payment, webhook, receipt, and email

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
PT-01	Start online payment	Customer with a Pending reservation of 600.00 chooses Online Payment.	Payment record is saved as Pending. 60,000 centavos is sent to PayMongo, and the Customer is sent to the PayMongo checkout page.	To be recorded after the actual test	Pending
PT-02	Successful payment (test mode)	Pay on the PayMongo page with a test card or test e-wallet.	Webhook sets the payment to Paid and the reservation to Reserved. Receipt is made and emailed.	To be recorded after the actual test	Pending

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
PT-03	Failed payment	Use a PayMongo test card that is declined.	Payment is set to Failed. The reservation is not confirmed, and no receipt is made.	To be recorded after the actual test	Pending
PT-04	Return page without paying	Open the return page of the website without paying on PayMongo.	The server asks PayMongo and finds no payment. The reservation stays Pending.	To be recorded after the actual test	Pending
PT-05	Webhook with a wrong signature	Send a webhook message with an invalid PayMongo signature.	The system answers 401. Nothing is changed.	To be recorded after the actual test	Pending
PT-06	Repeated webhook	Send the same valid webhook event two times.	The second event is ignored. The payment is not counted twice.	To be recorded after the actual test	Pending
PT-07	Webhook with a wrong amount	Send a valid, signed paid event whose amount is different from the reservation amount.	The reservation is not confirmed.	To be recorded after the actual test	Pending
PT-08	Webhook does not arrive	Paid successfully, but the webhook is blocked.	The scheduled job (APScheduler) checks PayMongo and confirms the reservation.	To be recorded after the actual test	Pending
PT-09	Download the receipt	Customer opens the history and downloads the receipt of a Paid reservation.	A PDF receipt is downloaded with the reference no., court, date, time, duration, amount, and status.	To be recorded after the actual test	Pending
PT-10	No receipt for unpaid reservations	Try to download a receipt for a Pending, unpaid, and cancelled reservation.	No receipt is made or shown for any of them.	To be recorded after the actual test	Pending

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
PT-11	Email failure	Stop the email service, then confirm a payment.	The reservation stays Reserved. The email is marked Failed and retried later. The receipt can still be downloaded.	To be recorded after the actual test	Pending

5.3 Security Test Cases

Table 7 lists the test cases for the security measures in Table 4. Each test tries to break a measure on purpose, and the system passes when the attempt is stopped.

Table 7

Security test cases

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
ST-01	Access to admin pages by a customer	Log in as a customer and open an admin page by typing its address.	Access is denied. The admin page is not shown.	To be recorded after the actual test	Pending
ST-02	Access without logging in	Open the reservation page without logging in.	The user is redirected to the login page.	To be recorded after the actual test	Pending
ST-03	Password storage	Open the Users table in the database after registration.	The password is saved as a hash, not as plain text.	To be recorded after the actual test	Pending
ST-04	SQL injection	Enter ' OR 1=1 -- as the email and password at login.	Login fails. The text is treated as normal input.	To be recorded after the actual test	Pending

Test ID	Test Case	Input / Steps	Expected Outcome	Actual Result	Status
ST-05	Cross-site scripting	Register with the name <script>alert (1) </script>.	The text is shown as plain text. No script runs.	To be recorded after the actual test	Pending
ST-06	Missing CSRF token	Send a form with the CSRF token removed.	The request is rejected. Nothing is saved.	To be recorded after the actual test	Pending
ST-07	Repeated login attempts	Enter the wrong password many times in a short time.	The system limits the attempts and shows a message to wait.	To be recorded after the actual test	Pending
ST-08	Changed price	Change the amount in the form (browser developer tools) before submitting.	The server computes the amount again. The saved amount is hourly rate x hours.	To be recorded after the actual test	Pending
ST-09	Session expiry	Log in, then wait until the session time ends.	The user is logged out and asked to log in again.	To be recorded after the actual test	Pending
ST-10	Card data	Complete an online payment, then check the Payments table.	No card number, card owner, or security code is stored.	To be recorded after the actual test	Pending
ST-11	Error pages	Open a page that does not exist and cause an error on purpose.	A generic error page is shown. No technical details are shown.	To be recorded after the actual test	Pending

5.4 Test Summary

Table 8 shows the number of test cases and the results after all tests were run. A test case is counted as Passed when the actual result is the same as the expected outcome. Any failed test is corrected and tested again.

Table 8

Summary of test results

Test group	Total	Passed	Failed	Pass rate
Functional test cases (Table 5)	23	TBD	TBD	TBD
Payment and receipt test cases (Table 6)	11	TBD	TBD	TBD
Security test cases (Table 7)	11	TBD	TBD	TBD
Total	45	TBD	TBD	TBD

CHAPTER 6

Conclusion and Future Recommendations

This chapter summarizes the objectives of the Private Sports Court Reservation System and how the design meets them, the challenges in joining the payment and email subsystems to the application, and the improvements that can be made in future versions.

6.1 Summary of Technical Objectives Met

The project aimed to give Customers a convenient way to reserve a private sports court in San Jose, Puerto Princesa City, and to give the Administrator one organized place to manage courts, reservations, and payments. The following list shows each specific objective from Chapter 1, how the system meets it, and the test cases in Chapter 5 that check it.

- **Availability Calendar showing Available, Reserved, and Pending courts.** Each court is shown per date and time with a status, so the Customer can check before booking (FR-2, SUC-2). (Related tests: TC-11.)
- **Reserve a court by date, time, and duration, and prevent double reservations.** The system rejects any overlap with a Reserved or Pending reservation. The court is locked during the check, so two requests at the same time cannot both succeed (FR-3, Section 4.1). (Related tests: TC-04, TC-06, TC-07.)
- **Online and on-site payment options.** Online payment uses PayMongo Checkout and is confirmed only after verification. On-site payment is held as Pending and confirmed by the Administrator (FR-4). (Related tests: PT-01 to PT-08, TC-08, TC-19.)
- **Confirmation and receipt sent to the registered email.** After a payment is confirmed, a PDF receipt is made and sent by Flask-Mail. A failed email is

marked and tried again (FR-5, Section 4.3). (Related tests: PT-02, PT-11, TC-19.)

- **Downloadable receipt right after payment.** The receipt can be downloaded from the reservation history, even if the email fails. No receipt is made for Pending, unpaid, or cancelled reservations (FR-5.4, FR-5.5, FR-6.2). (Related tests: PT-09, PT-10, PT-11.)
- **Administrator tools for courts, schedules, reservations, and payments in a secure database.** The Administrator can add and edit courts, mark them unavailable, update statuses, verify payments, and generate reports. Records are marked Cancelled, not deleted (FR-7 to FR-9). (Related tests: TC-09, TC-10, TC-14 to TC-23, ST-01 to ST-11.)

Together, these features are designed to replace manual checking of availability with a single platform. The Availability Calendar and the overlap check are meant to reduce double reservations; payment is confirmed by the system instead of by hand, and the receipt gives the Customer proof of reservation. The test cases in Chapter 5 check each of these features. Once the system is tested, a test case that does not give the expected outcome will be corrected and tested again before the system is accepted.

6.2 Challenges in Subsystem Integration

The most difficult parts of the project are the places where the application depends on another subsystem: the MySQL database, PayMongo, and the email service. The main challenges and how the design handles each one are listed below.

- **Double booking when two Customers reserve at the same time.** The court record is locked inside a database transaction while the overlap check runs. The reservation and its payment record are saved together or not at all.

- **Marking a reservation as paid only because the Customer reached the return page.** The return page does not trust the browser. The server asks PayMongo directly if the checkout session was paid. The webhook is the main way to confirm payment.
- **Fake or repeated webhook messages.** The HMAC-SHA256 signature of every webhook is checked, and a wrong signature gets a 401 answer. Each event ID is saved once, so a repeated event cannot confirm a reservation twice.
- **The webhook cannot reach a system that runs on a laptop.** A temporary public HTTPS URL is used during testing. A scheduled job (APScheduler) also checks pending online reservations with PayMongo, so a missed webhook does not leave a paid reservation unconfirmed.
- **An email problem undoing a confirmed payment.** The email is sent only after the database is saved. A failed email is marked Failed and tried again later, and the receipt can still be downloaded.
- **Wrong amounts and changed prices.** The server computes the amount from the court rate and hours each time. The amount sent to PayMongo is in centavos (600.00 x 100 = 60,000) and is checked again when the webhook arrives.
- **Unpaid on-site reservations blocking the schedule.** A scheduled job cancels unpaid Pending reservations when the hold time ends, and the schedule becomes Available again.

These integration points are checked by the payment and receipt test cases in Chapter 5 (PT-01 to PT-11). Problems that may appear during development, such as a webhook that does not arrive, an email setting that fails, or a database error, will be corrected and tested again with the same test cases.

6.3 Limitations

Status of the project. At the time of writing, the system is still under development and the test cases in Chapter 5 have not been run yet. For this reason, this chapter describes the design in Chapters 2 to 4 and what the tests are meant to check. It does not report test results. The Actual Result and Status columns in Chapter 5 are marked Pending and will be filled in after the actual tests.

The system also has limits that readers should keep in mind. Testing is planned to be done manually and with the PayMongo test mode, so no real money will be charged, and no test with many users at the same time is planned. The system runs on a local machine during development, which uses plain HTTP, and it serves one private sports facility. The security measures follow the OWASP and NIST guidance in Chapter 4, but a full security review by an outside party is not planned.

6.4 Architectural Improvements for Future Iterations

The current structure is simple on purpose, so that it is easy to build and maintain. The following improvements are recommended for future versions:

- **Host the system on a server with HTTPS.** Chapter 4 notes that a laptop uses plain HTTP. A production server with a certificate, a managed MySQL database, and automatic backups would protect data in transit and make the system always available.
- **Move the scheduled jobs to a separate worker.** The expiry check and payment check currently run with APScheduler inside the application. A separate job queue (for example, Celery with Redis) would keep them running even when the web application restarts, and would also handle email retries.

- **Add automated tests.** Testing in this project was manual. Automated tests for the reservation rules, the webhook, and the access control, run on every change, would catch problems early and make future updates safer.
- **Add logging and monitoring.** A central log and alerts for failed webhooks, failed emails, and server errors would help the Administrator find problems before Customers report them.
- **Add cancellation and refund rules.** The current design marks unpaid reservations as Cancelled, but it does not define refunds for paid reservations. A clear policy, supported by the PayMongo refund feature, would complete the payment cycle.
- **Add more notification channels.** SMS or in-app reminders before the reservation time would reduce no-shows and help Customers who do not check their email.
- **Support more courts, facilities, and staff roles.** The database already separates courts from reservations. Adding a facility table and a staff role with limited access would let the system grow beyond one court owner.
- **Improve the speed of the calendar.** If the number of reservations grows, database indexes on court, date, and time, and a short-term cache for the calendar, would keep the page fast.

6.5 Conclusion

The Private Sports Court Reservation System brings court reservation, payment, receipts, and administration into one organized platform. Its main design choices are the overlap check inside a transaction, payment confirmation by a signed webhook that is checked again by the server, and receipts that are made only after verified payment. These choices protect the data and prevent the problems of the

manual process: double bookings, delayed confirmations, and disorganized records. With HTTPS hosting, background jobs, automated tests, and the other improvements above, the system can grow from a single-facility prototype into a reliable service for private sports courts.

REFERENCES

International Organization for Standardization. (2023). ISO/IEC 25010:2023—Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE) Product quality model. https://www.iso.org/standard/78176.html

National Institute of Standards and Technology. (2022). Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (NIST Special Publication 800-218). https://doi.org/10.6028/NIST.SP.800-218

OWASP Foundation. (2025). Application Security Verification Standard (ASVS) version 5.0.0. https://owasp.org/www-project-application-security-verification-standard/

PCI Security Standards Council. (2024). Payment Card Industry Data Security Standard: Requirements and testing procedures (Version 4.0.1). https://www.pcisecuritystandards.org/standards/pci-dss/

World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/